

AI TRAINING – WEEK 2

DAY 3 ASSIGNMENT

File I/O, CSV, JSON, Exceptions, Modules & Packages

Read → Clean → Handle Errors → Export

1. What You Will Learn

You will learn how Python works with files and structured data. You will read CSV and JSON files, clean records, handle bad input safely, and export the results.

- File I/O – read and write files
- CSV – read tabular data
- JSON – read and write structured data
- Exceptions – handle errors using try/except
- Modules – reuse Python code from another file
- Packages – understand how related modules are organized
- Clean and validate data
- Export processed results

2. Student Deliverable – Keep It Simple

For this assignment, your main submission is ONE data processing script.

Main file: data_processing_YourName.py

Keep the sample CSV/JSON files used by your program in the same project folder.

- Read input data
- Clean the records
- Handle bad or missing data
- Process the data
- Export the cleaned result
- Use functions to keep the code organized

No screenshots or separate Word/PDF submission is required.

3. Part A – File I/O Basics

Create notes.txt using Python.

- Write three lines into the file.
- Read the file and print its contents.

```
file=open("notes.txt",'r')
data =file.read()
print(data)
file.close()
```

- Append one more line.

```
file=open("notes.txt",'a')
file.write('\n Hello again')
```

- Read the file again.

```
file=open("notes.txt",'a')
file.write('\n Hello again')
data =file.read()
print(data)
```

- Use the with statement when possible.

```
with open("notes.txt",'r') as file:
    text=file.read()
    print(text)
```

Remember: r = read, w = write, a = append.

4. Part B – Read a CSV File

Create students.csv with:

```
name,age,marks,city
Rahul,21,85,Hyderabad
Priya,22,92,Bangalore
Arjun,20,78,Chennai
```

- Open the CSV file.
- Read each record.
- Print each student's name and marks.

```
import csv
with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for student in reader:
        print(student["name"], student["marks"])
```

- Find the average marks.

```
import csv
total = 0
count = 0
with open("students.csv", "r") as file:
```

```

reader = csv.DictReader(file)

for student in reader:
    total += int(student["marks"])
    count += 1

average = total / count
print("Average marks:", average)

```

- Find the student with the highest marks.

```

import csv
highest_marks = 0
highest_student = ""
with open("students.csv", "r") as file:
    reader = csv.DictReader(file)

    for student in reader:
        marks = int(student["marks"])

        if marks > highest_marks:
            highest_marks = marks
            highest_student = student["name"]
print("Student:", highest_student)
print("Marks:", highest_marks)

```

5. Part C – Read JSON

Create students.json with:

```

[
  {"name": "Rahul", "marks": 85},
  {"name": "Priya", "marks": 92},
  {"name": "Arjun", "marks": 78}
]

```

- Read the JSON file.
- Print the student names.

```

import json

with open("student.json", "r") as file:
    students = json.load(file)
for student in students:
    print(student["name"])

```

- Calculate the average marks.

```
import json

with open("students.json", "r") as file:
    students = json.load(file)
total = 0
for student in students:
    total += student["marks"]
average = total / len(students)
print("Average marks:", average)
```

- Find the highest scorer.

```
import json
with open("students.json", "r") as file:
    students = json.load(file)
highest = students[0]
for student in students:
    if student["marks"] > highest["marks"]:
        highest = student

print("Highest scorer:", highest["name"])
print("Marks:", highest["marks"])
```

- Add one new student and save the updated JSON.

```
import json
with open("students.json", "r") as file:
    students = json.load(file)
new_student = {
    "name": "Ananya",
    "age": 21,
    "marks": 88
}

students.append(new_student)
with open("students.json", "w") as file:
    json.dump(students, file, indent=4)
print("New student added successfully")
```

6. Part D – Exceptions

Learn to handle bad input instead of allowing the program to crash.

- ValueError – invalid number conversion

- FileNotFoundError – file does not exist
- KeyError – dictionary key does not exist
- ZeroDivisionError – division by zero

Example:

try:

```
age = int(input("Enter age: "))
```

except ValueError:

```
print("Please enter a valid number.")
```

7. Part E – Modules

Create utils.py with small reusable functions such as:

- clean_name(name)
- calculate_average(numbers)
- is_valid_email(email)

```
from utils import clean_name, calculate_average, is_valid_email
name = clean_name(" devendra ")
print(name)
marks = calculate_average([80, 90, 70])
print("Average:", marks)
email = is_valid_email("devendra@gmail.com")
print("Valid email:", email)
```

output:

Devendra

Average: 80.0

Valid email: True

Create another Python file and import these functions. The goal is to understand how code can be split into reusable modules.

8. Part F – Packages

Understand the basic idea of a package: a folder containing related Python modules.

Example structure:

project/

main.py

utils/

__init__.py

cleaning.py

validation.py

You do not need to build a complex package. Focus on understanding the structure.

9. Part G – Quick Debugging

Find the problem, fix it, run the code, and explain the mistake.

Debugging Problem 1

```
file = open("students.csv", "r")
content = file.read()
print(content)
```

Task: identify the error and handle/fix it safely.

Ans: there is no major bug in the program but we want to close the file.

Debugging Problem 2

```
age = int("twenty")
```

ans: value error because twenty cannot be converted into integer.

```
try:
    age = int("twenty")
except ValueError:
    print("Invalid age")
```

Task: identify the error and handle/fix it safely.

Debugging Problem 3

```
student = {"name": "Rahul"}
print(student["marks"])
```

ans: error because there are no items called marks in the dictionary.

Task: identify the error and handle/fix it safely.

Debugging Problem 4

```
result = 10 / 0
```

ans: it is a ZeroDivisionError because anything by zero is infinite.

Task: identify the error and handle/fix it safely.

10. Part H – Main Hands-on Activity: Data Processing Script

Build a simple data processing script that reads student records, cleans the data, handles bad records, and exports the cleaned result.

Use a CSV file such as:

```
name,age,marks,city
Rahul,21,85,Hyderabad
Priya,22,92,Bangalore
,20,75,Chennai
Arjun,abc,80,Chennai
Sneha,21,95,
Kiran,20,88,Hyderabad
```

Your program should:

- Read the CSV file.

```
import csv
with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for student in reader:
        print(student)
```

- Check for missing names or cities.

```
import csv
with open("students.csv", "r") as file:
    reader = csv.DictReader(file)

    for student in reader:
        name = student["name"].strip()
        city = student["city"].strip()

        if name == "" or city == "":
            print("Missing name or city:", student)
```

- Handle invalid age/marks values without crashing.

```
import csv
with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for student in reader:
        try:
            age = int(student["age"])
            marks = float(student["marks"])

            print(student["name"], age, marks)
        except ValueError:
            print("Invalid age or marks:", student)
```

- Clean valid records.

```
import csv
valid_students = []
with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for student in reader:
        try:
            name = student["name"].strip()
            city = student["city"].strip()
```

```

        age = int(student["age"])
        marks = float(student["marks"])

        if name == "" or city == "":
            continue

        valid_students.append({
            "name": name,
            "age": age,
            "marks": marks,
            "city": city
        })
    except ValueError:
        continue

print(valid_students)

```

- Ignore or separately report invalid records.

```

total = 0

for student in valid_students:
    total += student["marks"]

average = total / len(valid_students)

print("Average marks:", average)

```

- Calculate the average marks of valid students.

```

highest = valid_students[0]

for student in valid_students:
    if student["marks"] > highest["marks"]:
        highest = student

print("Highest scorer:", highest["name"])
print("Marks:", highest["marks"])

```

- Find the highest scorer.

```

import csv
with open("cleaned_students.csv", "w", newline="") as file:
    writer = csv.DictWriter(
        file,
        fieldnames=["name", "age", "marks", "city"]
    )

```

```

writer.writeheader()
writer.writerows(valid_students)

print("Valid records saved successfully")

```

- Export valid records into cleaned_students.csv.

```

import csv

invalid_students = []

with open("students.csv", "r") as file:
    reader = csv.DictReader(file)

    for student in reader:
        try:
            name = student["name"].strip()
            city = student["city"].strip()
            age = int(student["age"])
            marks = float(student["marks"])

            if name == "" or city == "":
                invalid_students.append(student)
                continue

        except ValueError:
            invalid_students.append(student)

with open("errors.csv", "w", newline="") as file:
    writer = csv.DictWriter(
        file,
        fieldnames=["name", "age", "marks", "city"]
    )

    writer.writeheader()
    writer.writerows(invalid_students)

print("Invalid records saved successfully")

```

- Save invalid records into errors.csv or display them clearly.

11. Suggested Program Structure

- read_csv() – read the input file
- clean_record() – clean and validate one record
- process_records() – process valid records
- export_csv() – save cleaned data

- `main()` – control the complete program

Bonus: put reusable cleaning or validation functions into `utils.py`.

12. Part I – Explain in Your Own Words

1. What is file I/O?

Answer: file I/O is nothing but reading and writing the file using program.

2. What is the difference between CSV and JSON?

Answer: CSV stores the data in rows and columns but JSON stores like key values pairs.

3. Why do we use try/except?

Answer: we use try/except to continue the program execution without stopping in middle.even errors occurred.

4. What is an exception?

Answer: excetion is error occurred in the middle of the program.

5. What is a Python module?

Answer: which contains reusable functions.

6. What is a package?

Answer: which contains python related modules.

7. Why should we validate data before processing it?

Answer: to avoid error while processing the data.

8. Why is it better to split a large program into functions?

Answer: makes programs easier to understand.

13. Daily Deliverable

Submit your working data processing script.

File name: `data_processing_YourName.py`

Your script should demonstrate:

- CSV reading
- Data cleaning
- Exception handling
- Data processing
- CSV export
- Functions
- Clear and readable code

14. Student Submission Checklist

- I can read a text file.
- I can read a CSV file.
- I can read and write JSON.
- I can handle common exceptions.
- I understand modules and packages.
- I cleaned invalid data instead of allowing the program to crash.
- I exported the processed data.
- I used functions to organize my program.
- I tested my program with bad input.
- I can explain the important concepts in my own words.

15. After This Assignment, You Should Be Able To

- Work with files using Python.
- Read and write CSV and JSON data.
- Handle common runtime errors safely.
- Create and import simple modules.
- Understand the purpose of packages.
- Clean and validate real-world data.
- Build a small end-to-end data processing script.

Key Principle: Real-world data is rarely perfect. Good programs read it, validate it, handle errors, and produce clean results.